



Security Review Report for Thesauros

July 2026



Table of Contents

1. About Hexens
2. Executive summary
3. Security Review Details
 - Security Review Lead
 - Scope
 - Changelog
4. Severity Structure
 - Severity characteristics
 - Issue symbolic codes
5. Findings Summary
6. Weaknesses
 - Fixed-order withdrawals may fail despite available liquidity
 - Persistent MetaMorpho `lostAssets` may misstate Rebalancer position value
 - Provider adapters hardcode arbitrum protocol addresses

1. About Hexens

Hexens is a pioneering cybersecurity firm dedicated to establishing robust security standards for Web3 infrastructure, driving secure mass adoption through innovative protection technology and frameworks. As an industry elite experts in blockchain security, we deliver comprehensive audit solutions across specialized domains, including infrastructure security, Zero Knowledge Proof, novel cryptography, DeFi protocols, and NFTs.

Our methodology combines industry-standard security practices combined with unique methodology of two teams per audit, continuously advancing the field of Web3 security. This innovative approach has earned us recognition from industry leaders.

Since our founding in 2021, we have built an exceptional portfolio of enterprise clients, including major blockchain ecosystems and Web3 platforms.

2. Executive Summary

This report covers the security review for Thesauros, a DeFi protocol for automated interest rebalancing. It supports ERC4626 and uses different liquidity providers to maximize yield. This engagement covered an iterative code update to the protocol.

Our security assessment was a full review of the new code, spanning a total of 3 days.

During our review, we did not identify any major severity vulnerabilities.

We did identify some minor severity vulnerabilities and code optimizations.

All reported issues were fixed by the development team and subsequently verified by us.

We can confidently say that the overall security and code quality have increased after completion of our audit.

3. Security Review Details

- **Review Led by**

Jahyun Koo, Lead Security Researcher

- **Scope**

The analyzed resources are located on:

<https://github.com/Thesauros/optimized-rebalancer-contracts/commit/fd1c8747a01732448774ca3bfd6bc6cf2024b5fc>

The issues described in this report were fixed in the following commit:

<https://github.com/Thesauros/optimized-rebalancer-contracts/tree/f970dcc77385468e337a9c54224f39ef0fd29a82>

- **Changelog**

■ 14 July 2026	Audit start
■ 20 July 2026	Initial report
■ 23 July 2026	Revision received
■ 31 July 2026	Final report

4. Severity Structure

The vulnerability severity is calculated based on two components:

- 1. Impact of the vulnerability
- 2. Probability of the vulnerability

Impact	Probability			
	Rare	Unlikely	Likely	Very likely
Low	Low	Low	Medium	Medium
Medium	Low	Medium	Medium	High
High	Medium	Medium	High	Critical
Critical	Medium	High	Critical	Critical

▪ Severity Characteristics

Smart contract vulnerabilities can range in severity and impact, and it's important to understand their level of severity in order to prioritize their resolution. Here are the different types of severity levels of smart contract vulnerabilities:

Critical	Vulnerabilities that are highly likely to be exploited and can lead to catastrophic outcomes, such as total loss of protocol funds, unauthorized governance control, or permanent disruption of contract functionality.
High	Vulnerabilities that are likely to be exploited and can cause significant financial losses or severe operational disruptions, such as partial fund theft or temporary asset freezing.

Medium

Vulnerabilities that may be exploited under specific conditions and result in moderate harm, such as operational disruptions or limited financial impact without direct profit to the attacker.

Low

Vulnerabilities with low exploitation likelihood or minimal impact, affecting usability or efficiency but posing no significant security risk.

Informational

Issues that do not pose an immediate security risk but are relevant to best practices, code quality, or potential optimizations.

▪ Issue Symbolic Codes

Each identified and validated issue is assigned a unique symbolic code during the security research stage.

Due to the structure of the vulnerability reporting flow, some rejected issues may be missing.

5. Findings Summary

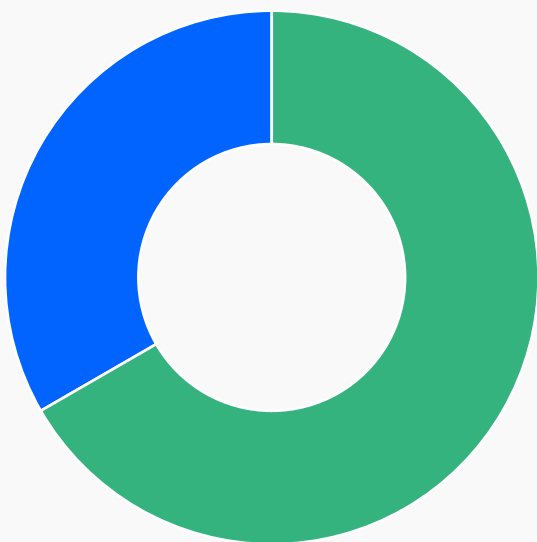
Severity

Number of findings

Critical	0
High	0
Medium	0
Low	2
Informational	1

Total:

3



Low

Informational



Fixed

6. Weaknesses

This section contains the list of discovered weaknesses.

THES2-1 | Fixed-order withdrawals may fail despite available liquidity

Fixed 

Severity:

Low

Probability:

Rare

Impact:

Low

Path:

contracts/Rebalancer.sol#L447

Description:

The withdrawal flow treats each provider's reported accounting balance as immediately withdrawable liquidity and processes providers in a fixed order. If an earlier provider cannot satisfy the requested amount and reverts, the transaction stops before later providers are considered.

This can occur with **MorphoProvider**, where **getDepositBalance()** returns the asset value of the vault's **MetaMorpho** shares. This value may remain positive while borrowing activity reduces the assets immediately available for withdrawal. Consequently, a withdrawal may revert even when a later provider holds sufficient liquidity.

```
uint256 assetsAtProvider = provider.getDepositBalance(
    address(this),
    this
);

uint256 amount = (assetsAtProvider >= assetsLeft)
    ? assetsLeft
    : assetsAtProvider;

_delegateActionToProvider(amount, "withdraw", provider);
```

- MetaMorphoV1_1.sol

```
function _withdrawMorpho(uint256 assets) internal {
```

```

for (uint256 i; i < withdrawQueue.length; ++i) {
    Id id = withdrawQueue[i];
    MarketParams memory marketParams = _marketParams(id);
    (uint256 supplyAssets, Market memory market) = _accruedSupplyBalance(marketParams, id);

    uint256 toWithdraw = UtilsLib.min(
        _withdrawable(marketParams, market.totalSupplyAssets, market.totalBorrowAssets, supplyAssets),
assets
    );

    if (toWithdraw > 0) {
        // Using try/catch to skip markets that revert.
        try MORPHO.withdraw(marketParams, toWithdraw, 0, address(this), address(this)) {
            assets -= toWithdraw;
        } catch {}
    }

    if (assets == 0) return;
}

if (assets != 0) revert ErrorsLib.NotEnoughLiquidity();
}

```

The failed transaction restores the burned shares and accounting state, but the user may remain unable to withdraw until liquidity returns or the provider order is changed.

Remediation:

- Use immediately withdrawable liquidity instead of accounting balances when determining each provider's withdrawal amount.
- Isolate failed provider withdrawals and continue processing the remaining providers.
- Calculate the remaining amount from the actual underlying tokens received from each provider.
- Revert only after all providers have been attempted and the requested amount remains unavailable.

THES2-3 | Persistent MetaMorpho `lostAssets` may misstate Rebalancer position value

Fixed 

Severity:

Low

Probability:

Rare

Impact:

Low

Path:

contracts/providers/AaveV3Provider.sol

Description:

MetaMorpho V1.1 records assets missing through realized bad debt or forced market removal as **lostAssets** and continues to include them in **totalAssets()**. The update rule only increases or preserves **lostAssets**; once positive, it cannot return to zero through normal vault operations.

- MetaMorphoV1_1.sol

```
function _accruedFeeAndAssets()
    internal
    view
    returns (uint256 feeShares, uint256 newTotalAssets, uint256 newLostAssets)
{
    uint256 realTotalAssets;
    for (uint256 i; i < withdrawQueue.length; ++i) {
        realTotalAssets += MORPHO.expectedSupplyAssets(_marketParams(withdrawQueue[i]),
address(this));
    }

    if (realTotalAssets < lastTotalAssets - lostAssets) newLostAssets = lastTotalAssets - realTotalAssets;
    else newLostAssets = lostAssets;
    ...
}
```

Morpho's documentation explains that V1.1 does not immediately allocate realized bad debt across all depositors. Unless the loss is covered, the remaining depositors may eventually face the liquidity shortfall. (Ref, [Managing Bad Debt: How-to Guide](#))

MorphoProvider uses **MetaMorpho**'s book conversion as the reported position balance. Its rate calculation weights the real assets supplied to Morpho markets but uses the same **lostAssets**-inclusive book value as its denominator.

```

function getDepositBalance(
    address user,
    IRebalancer
) external view returns (uint256 balance) {
    uint256 shares = _metaMorpho.balanceOf(user);
    balance = _metaMorpho.convertToAssets(shares);
}

function getDepositRate(
    IRebalancer
) external view returns (uint256 rate) {
    IMorpho morpho = _getMorpho();

    uint256 ratio;
    uint256 queueLength = _metaMorpho.withdrawQueueLength();

    uint256 totalDeposits = _metaMorpho.totalAssets();

    for (uint256 i; i < queueLength; i++) {
        Id idMarket = _metaMorpho.withdrawQueue(i);

        MarketParams memory marketParams = morpho.idToMarketParams(
            idMarket
        );
        Market memory market = morpho.market(idMarket);

        uint256 marketRate = _getMarketRate(marketParams, market);
        uint256 assetsInMarket = morpho.expectedSupplyAssets(
            marketParams,
            address(_metaMorpho)
        );
        ratio += marketRate.wMulDown(assetsInMarket);
    }

    rate =
        ratio.mulDivDown(1e18 - _metaMorpho.fee(), totalDeposits) *
        10 ** 9;
}

```

Following a material realized loss that has not been sufficiently covered, the **Rebalancer** may value its **MetaMorpho** position above the recoverable backing. If sufficient liquidity remains in **MetaMorpho** or another listed provider, an earlier redemption may be settled at the book

valuation, leaving more of the loss to later shareholders.

Remediation:

- Implement a shared accounting routine that derives the **Rebalancer**'s recoverable **MetaMorpho** position from current real assets and exercisable share claims.
- Use the same accounting basis in **getDepositBalance()** and **getDepositRate()**.
- Ensure **Rebalancer** share conversions, fee calculations, and withdrawals consume the adjusted provider balance.

THES2-2 | Provider adapters hardcode arbitrum protocol addresses

Fixed 

Severity:

Informational

Probability:

Unlikely

Impact:

Informational

Path:

contracts/providers/AaveV3Provider.sol#L42
contracts/providers/DolomiteProvider.sol#L49
contracts/providers/RevertProvider.sol#L33

Description:

The Aave V3, Dolomite, and Revert providers embed Arbitrum protocol addresses directly in their implementations. These addresses are used without verifying the current chain or whether the targets contain the expected contract code.

```
// AaveV3Provider.sol
return IPoolAddressesProvider(
    0xa97684ead0e402dC232d5A977953DF7ECBaB3CDb
);

// DolomiteProvider.sol
return IDolomiteMargin(
    0x6Bd780E7fDf01D77e4d475c821f1e7AE05409072
);
return IDolomiteGetter(
    0x9381942De7A66fdB4741272EaB4fc0A362F7a16a
);
return IDepositWithdrawalProxy(
    0xAdB9D68c613df4AA363B42161E1282117C7B9594
);

// RevertProvider.sol
return IV3Vault(
    0x74E6AFef5705BEb126C6d3Bf46f8fad8F3e07825
);
```

When the adapters are used on Base, these addresses do not resolve to the expected protocol contracts. Provider registration does not validate the source bytecode or interface before granting approval and storing the provider list.

Remediation:

- Replace embedded protocol addresses with validated chain-specific configuration.
- Validate target bytecode and expected interface responses before storing providers or granting approvals.
- Complete validation of all providers before committing a provider-list update, and revoke approvals for removed sources.

hexens x  Thesauros

